

유전 알고리즘(Genetic Algorithm, GA)은 자연 선택과 유전의 원리를 모방한 최적화 및 탐색 기법입니다. 이는 다윈의 자연선택(Natural Selection) 이론을 기반으로 하여, 문제 해결을 위해 가능한 해를 진화 과정과 비슷한 방식으로 찾아가는 방법입니다. 유전 알고리즘은 주로 복잡한 문제나 해 공간이 매우 넓은 경우에 사용되며, 다음과 같은 주요 요소들로 구성됩니다:

1. 개체(Chromosome)

유전 알고리즘에서의 개체는 가능한 해(solution)를 나타냅니다. 이 개체는 유전자(Genes)로 구성된 문자열로 표현되며, 보통 이진수(0과 1) 혹은 숫자 배열로 나타냅니다. 각 개체는 문제의 해를 나타내는 특성을 갖습니다.

2. 초기 집단(Population)

초기 집단은 문제에 대한 여러 개체들의 집합으로, 초기 해의 무리를 구성합니다. 유전 알고리즘은 한 번에 여러 개체를 처리하여 병렬로 탐색을 수행합니다. 초기 집단은 주로 무작위로 생성되며, 각 개체는 서로 다른 해를 표현합니다.

3. 적합도 함수(Fitness Function)

적합도 함수는 각 개체의 성능을 평가하는 함수입니다. 이 함수는 개체가 얼마나 좋은 해를 제공하는지 평가하며, 목표는 적합도 값이 높은 해를 찾는 것입니다. 적합도 함수는 문제의 특성에 따라 설계되며, 최대화 혹은 최소화 문제에 따라 적합도를 평가합니다.

4. 선택(Selection)

선택은 적합도가 높은 개체들이 다음 세대로 전달되기 위해 선택되는 과정입니다. 적합도가 높을수록 선택될 확률이 커지며, 이를 통해 우수한 개체들이 자손을 남길 수 있게 됩니다. 대표적인 선택 방법으로는 룰렛 휠 선택(Roulette Wheel Selection), 순위 기반 선택(Rank Selection) 등이 있습니다.

5. 교차(Crossover)

교차는 두 개체 간의 유전자 정보를 교환하는 과정으로, 이를 통해 새로운 자손이 생성됩니다. 두 개체의 유전자를 서로 교환하여 새로운 조합을 만들어내는 과정입니다. 교차는 자손들이 부모 세대보다 더 나은 해를 가질 가능성을 높입니다. 단일 교차(one-point crossover), 다중 교차(multi-point crossover) 등이 일반적으로 사용됩니다.

6. 돌연변이(Mutation)

돌연변이는 개체의 유전자를 무작위로 변경하는 과정입니다. 이는 다양성을 유지하고, 지역 최적해에 갇히는 것을 방지하기 위해 사용됩니다. 돌연변이율은 보통 낮게 설정되며, 유전자의 일부를 무작위로 변형하여 새로운 탐색 경로를 개척합니다.

7. 종료 조건(Termination)

유전 알고리즘은 특정 조건을 만족하면 종료됩니다. 종료 조건은 다음과 같이 설정될 수 있습니다:

- 최대 세대 수에 도달한 경우
- 적합도 값이 일정 수준 이상인 경우
- 해의 변화가 일정 세대 동안 없을 경우

8. 과정 요약

유전 알고리즘은 다음과 같은 순서로 진행됩니다:

1. 초기 집단 생성
2. 적합도 평가
3. 적합도가 높은 개체들 선택
4. 교차 및 돌연변이를 통해 자손 생성
5. 새로운 세대로 업데이트
6. 종료 조건을 만족할 때까지 반복

유전 알고리즘의 장점

- 복잡한 문제의 해를 탐색하는 데 효과적이며, 다중 전역 최적해를 탐색할 수 있습니다.
- 해 공간이 매우 크거나, 함수가 미분 불가능한 경우에도 사용 가능합니다.
- 병렬 처리가 가능해 빠른 탐색이 가능합니다.

유전 알고리즘의 단점

- 적합한 적합도 함수와 매개변수 설정이 필요합니다.
- 지역 최적해에 빠질 가능성이 있으며, 이를 방지하기 위해 돌연변이 비율 등을 신중히 설정해야 합니다.

유전 알고리즘은 최적화 문제, 기계 학습, 로봇 공학 등 다양한 분야에서 활용되며, 복잡한 문제를 효율적으로 해결하기 위해 자주 사용됩니다.

여행자 문제 (TSP, 최단거리 계산)

유전 알고리즘(Genetic Algorithm, GA)을 이용하여 여행자 문제(Traveling Salesman Problem, TSP)의 최단 경로를 계산할 수 있습니다. TSP는 매우 복잡한 NP-완전 문제로, 도시들의 집합과 각 도시 간의 거리가 주어졌을 때, 한 번씩만 방문하고 출발 도시로 돌아오는 최단 경로를 찾는 문제입니다.

유전 알고리즘은 이 문제의 해를 탐색하는 데 효과적인 방법 중 하나로, 다양한 해를 병렬로 탐색하고, 진화를 통해 점차 최적 또는 근사 최적해를 찾을 수 있습니다.

1. 문제 설정

- 개체(Chromosome): 개체는 도시들을 방문하는 순서를 나타내는 배열로 표현됩니다. 예를 들어, 도시가 5개 있다면 개체는 [1, 3, 2, 5, 4]와 같은 형태가 될 수 있습니다. 이는 첫 번째 도시에 시작하여 순서대로 방문하는 경로를 의미합니다.

- 적합도 함수(Fitness Function): 적합도는 경로의 총 거리로 계산됩니다. 적합도 함수는 거리가 짧을수록 높은 적합도를 부여하여, 짧은 경로를 가진 개체들이 다음 세대에 더 많이 선택되도록 유도합니다. 목표는 경로의 총 거리를 최소화하는 것입니다.

2. 유전 알고리즘의 단계

1. 초기 집단 생성: 가능한 경로(도시 방문 순서)를 무작위로 생성하여 초기 집단을 만듭니다. 이 집단은 여러 개체로 구성되며, 각 개체는 특정 경로를 나타냅니다.

2. 적합도 평가: 각 개체의 경로 길이를 계산하고, 이를 적합도 함수에 따라 평가합니다. 적합도가 높은 개체는 더 짧은 경로를 의미하며, 이 개체들이 다음 세대에서 더 높은 선택 확률을 갖습니다.

3. 선택(Selection): 적합도에 따라 부모 개체를 선택합니다. 적합도가 높은 개체는 다음 세대에서 교차 및 돌연변이의 대상이 됩니다. 룰렛 휠 선택이나 토너먼트 선택과 같은 방법을 사용할 수 있습니다.

4. 교차(Crossover): 두 개체 간 교차를 통해 새로운 자손을 생성합니다. 순열 특성(모든 도시는 한 번만 방문)을 유지해야 하므로, 순열 교차 연산자(Order Crossover, OX)와 같은 방법을 사용할 수 있습니다. 예를 들어, 부모 개체들의 일부 순서를 자손에 그대로 전달하고, 나머지는 부모 개체에서 중복되지 않도록 채워 넣습니다.

5. 돌연변이(Mutation): 돌연변이는 개체의 유전자(도시 방문 순서)를 변경하여 새로운 경로를 만듭니다. 예를 들어, 두 개의 도시를 무작위로 선택하여 서로 위치를 바꾸는 방식으로 돌연변이를 적용할 수 있습니다. 이를 통해 다양성을 확보하고, 지역 최적해에 빠지지 않도록 도와줍니다.

6. 종료 조건: 최대 세대 수에 도달하거나, 적합도(경로 길이)가 일정 수준 이하로 변하지 않으면 알고리즘이 종료됩니다.

3. 장점

- 유전 알고리즘은 여러 개의 해를 동시에 탐색하므로, 지역 최적해에 빠지지 않고 전역 최적해를 찾을 가능성이 높습니다.
- 교차 및 돌연변이 연산을 통해 다양한 경로를 탐색할 수 있어, 특히 복잡한 문제에서 유용합니다.

4. 단점

- 최적해를 보장하지 않으며, 근사 해를 찾는 방식입니다. 하지만 충분히 많은 세대를 거치면 상당히 좋은 근사 해를 찾을 수 있습니다.
- 알고리즘의 매개변수(인구 크기, 돌연변이율, 교차율 등) 설정이 성능에 크게 영향을 미칩니다.

5. TSP에 유전 알고리즘을 적용한 과정 예시

- Step 1: 10개의 도시에 대한 랜덤한 경로들을 초기 집단으로 설정합니다.
- Step 2: 각 경로에 대해 적합도를 계산하여, 짧은 경로일수록 적합도가 높아지도록 설정합니다.
- Step 3: 선택 과정에서 적합도가 높은 경로를 가진 개체들이 자손을 만들도록 선택합니다.
- Step 4: 교차 연산을 통해 새로운 경로를 생성합니다.
- Step 5: 일정 확률로 돌연변이를 발생시켜, 두 도시간의 순서를 바꾸는 방식으로 변이를 추가합니다.
- Step 6: 세대가 계속됨에 따라 점차 경로가 짧아지고, 종료 조건에 도달할 때까지 반복합니다.

<R example>

필요한 패키지 설치

```
install.packages("GA")
```

```
library(GA)
```

10개의 도시 사이의 거리 매트릭스를 정의

```
distance_matrix <- matrix(c(
  0, 29, 20, 21, 16, 31, 100, 12, 4, 31,
  29, 0, 15, 29, 28, 40, 72, 21, 29, 41,
  20, 15, 0, 15, 14, 25, 81, 9, 23, 27,
  21, 29, 15, 0, 4, 12, 92, 12, 25, 13,
  16, 28, 14, 4, 0, 16, 94, 9, 20, 16,
  31, 40, 25, 12, 16, 0, 95, 24, 36, 3,
  100, 72, 81, 92, 94, 95, 0, 90, 101, 99,
  12, 21, 9, 12, 9, 24, 90, 0, 15, 25,
  4, 29, 23, 25, 20, 36, 101, 15, 0, 35,
  31, 41, 27, 13, 16, 3, 99, 25, 35, 0
), nrow = 10, byrow = TRUE)
```

경로의 총 거리를 계산하는 적합도 함수 정의

```
tsp_fitness <- function(order) {
  tour <- c(order, order[1]) # 순회 경로 생성
  distance <- 0
  for (i in 1:(length(order))) {
    distance <- distance + distance_matrix[tour[i], tour[i + 1]]
  }
  return(-distance) # GA는 적합도를 최대화하려고 하므로, 거리는 음수로 반환
}
```

GA 매개변수 설정 및 실행

```
ga_tsp <- ga(
  type = "permutation", # 순열 문제
  fitness = tsp_fitness, # 적합도 함수
  lower = 1, # 도시 번호 최소값
  upper = 10, # 도시 번호 최대값
  popSize = 50, # 개체 수 (population size)
  maxiter = 500, # 최대 세대 수
  run = 100, # 적합도가 변하지 않는 세대 수에 도달하면 종료
```

```
    pmutation = 0.2, # 돌연변이 확률  
    elitism = 2 # 엘리트 개체 수  
  )
```

```
# 결과 출력  
summary(ga_tsp)
```

```
# 최적 경로 출력  
best_order <- ga_tsp@solution[1,]  
best_order
```

```
# 최적 경로의 거리 계산  
best_distance <- -tsp_fitness(best_order)  
best_distance
```

Simulated Annealing (SA)은 최적화 문제를 해결하기 위한 메타휴리스틱 알고리즘 중 하나로, 전역 최적해를 찾기 위해서 주로 사용됩니다. 이 알고리즘은 금속을 서서히 냉각하여 결정 구조를 안정화시키는 물리화학적 공정인 담금질(annealing)에서 영감을 받아 개발되었습니다.

핵심 아이디어

Simulated Annealing의 기본 개념은 지역 최적해에 빠지지 않고 전역 최적해를 찾는 것입니다. 일반적인 탐색 알고리즘은 현재 상태에서 더 좋은 해로만 이동하기 때문에 지역 최적해에 빠질 위험이 큼니다. 그러나 SA는 탐색 과정에서 일시적으로 나쁜 해로 이동할 수 있도록 허용하여, 다양한 해를 탐색하고 전역 최적해를 찾을 가능성을 높입니다.

주요 개념

1. 현재 상태(Current State): 현재 탐색하고 있는 해를 의미합니다.
2. 이웃 상태(Neighbor State): 현재 상태에서 약간 변화된 해입니다. 이웃 상태는 주로 작은 무작위 변화를 통해 생성됩니다.
3. 에너지(Energy): 주어진 해의 품질을 나타내는 값입니다. 대부분 최적화 문제에서 에너지는 목적 함수(objective function) 값과 동일합니다.
4. 온도(Temperature): 초기에는 높게 설정되고, 시간이 지남에 따라 점진적으로 감소하는 변수입니다. 온도는 알고리즘의 탐색 범위를 제어합니다. 초기에는 온도가 높기 때문에 나쁜 해로도 이동할 수 있지만, 온도가 낮아질수록 좋은 해로만 이동하려고 합니다.

알고리즘 과정

Simulated Annealing의 동작은 다음과 같은 단계를 거칩니다:

1. 초기 상태 설정: 임의의 초기 해를 설정하고, 이 해의 에너지를 계산합니다. 초기 온도도 설정합니다.
2. 이웃 상태 생성: 현재 상태에서 무작위로 이웃 상태를 생성합니다.
3. 에너지 계산: 이웃 상태의 에너지를 계산하여 현재 상태와 비교합니다.
4. 이동 여부 결정:
 - 더 좋은 해(에너지가 더 낮은 경우): 이웃 상태가 현재 상태보다 에너지가 낮으면, 새로운 해로 이동합니다.
 - 더 나쁜 해(에너지가 더 높은 경우): 에너지가 더 높더라도, 확률적으로 나쁜 해로 이동할 수 있습니다. 이 확률은 아래의 메트로폴리스 기준(Metropolis criterion)을 따릅니다:

$$P = \exp\left(\frac{-\Delta E}{T}\right)$$

여기서 ΔE 는 에너지 차이(이웃 상태 에너지 - 현재 상태 에너지)이고, T 는 현재 온도입니다. 이 확률은 온도가 높을수록 커지며, 온도가 낮아지면 나쁜 해로 이동할 가능성은 줄어듭니다.

5. 온도 감소: 일정한 방식으로 온도를 감소시킵니다. 일반적으로 냉각 스케줄을 따라 온도가 서서히 줄어들게 설정합니다.

6. 종료 조건 확인: 종료 조건(최대 반복 횟수, 온도가 너무 낮아지거나 에너지가 더 이상 개선되지 않는 경우 등)을 만족하면 알고리즘을 종료합니다. 그렇지 않으면 2단계로 돌아가서 다시 반복합니다.

메트로폴리스 기준

SA에서 나쁜 해로 이동할 수 있는 확률은 메트로폴리스 기준에 의해 결정됩니다. 이 기준은 에너지 차이 ΔE 가 작을수록, 그리고 온도 T 가 높을수록 나쁜 해로 이동할 확률을 높게 설정합니다. 이는 탐색 초기에 다양한 해를 탐색할 수 있도록 도와주며, 이후 온도가 낮아질수록 탐색 범위를 좁혀 최적해에 도달할 수 있도록 합니다.

알고리즘 요약

1. 초기 해 설정 및 초기 온도 설정
2. 이웃 상태 생성
3. 이웃 상태와 현재 상태의 에너지 비교
4. 더 좋은 해로 이동하거나, 확률적으로 나쁜 해로 이동
5. 온도를 서서히 낮추며 이 과정을 반복
6. 종료 조건에 도달하면 최적해 반환

Simulated Annealing의 장점

1. 전역 최적해 탐색 가능성: 탐색 과정에서 나쁜 해로 이동할 수 있기 때문에 지역 최적해에 빠지지 않고 전역 최적해를 탐색할 가능성이 높습니다.
2. 단순함: SA는 구현이 비교적 간단하며, 다양한 최적화 문제에 적용할 수 있습니다.
3. 미분 정보 불필요: 함수의 미분 가능 여부에 상관없이 사용할 수 있습니다. 비선형, 비미분 함수에도 적용 가능합니다.

Simulated Annealing의 단점

1. 시간 소모: 냉각 스케줄을 잘못 설정하거나 반복 횟수가 많아지면 최적해를 찾는 데 오랜 시간이 걸릴 수 있습니다.
2. 파라미터 설정: 적절한 초기 온도와 냉각 스케줄을 설정하는 것이 중요하며, 문제에 따라 그 설정이 다를 수 있습니다.
3. 전역 최적해 보장 없음: 전역 최적해에 도달할 가능성은 높지만, 항상 보장되지는 않습니다.

Simulated Annealing의 적용 분야

SA는 다양한 최적화 문제에 사용할 수 있으며, 특히 아래와 같은 문제에 적합합니다:

- 비선형 최적화 문제
- 비미분 함수의 최적화
- 조합 최적화 문제 (예: 여행자 문제(TSP), 배낭 문제)
- 기계 학습의 하이퍼파라미터 최적화
- 물리학적 모델링에서 에너지 최적화

예시: Simulated Annealing을 이용한 TSP 해결

SA를 이용하여 여행자 문제(Traveling Salesman Problem, TSP)를 해결하는 간단한 과정은 다음과 같습니다:

1. 도시 간의 거리 행렬을 설정합니다.
2. 초기 경로를 무작위로 생성합니다.
3. 이웃 경로는 현재 경로에서 두 도시의 위치를 바꾸는 방식으로 생성됩니다.
4. 경로의 총 길이를 에너지로 간주하고, SA 알고리즘을 통해 점점 짧은 경로를 찾습니다.

Simulated Annealing은 전역 최적화 문제에 대한 훌륭한 해법을 제공할 수 있으며, 함수의 성질에 관계없이 적용할 수 있는 유용한 방법입니다.

```
# 필요한 패키지 설치
```

```
# install.packages("GenSA")
```

```
library(GenSA)
```

```
# 10개의 도시 사이의 거리 매트릭스를 정의
```

```
distance_matrix <- matrix(c(
  0, 29, 20, 21, 16, 31, 100, 12, 4, 31,
  29, 0, 15, 29, 28, 40, 72, 21, 29, 41,
  20, 15, 0, 15, 14, 25, 81, 9, 23, 27,
  21, 29, 15, 0, 4, 12, 92, 12, 25, 13,
```

```

16, 28, 14, 4, 0, 16, 94, 9, 20, 16,
31, 40, 25, 12, 16, 0, 95, 24, 36, 3,
100, 72, 81, 92, 94, 95, 0, 90, 101, 99,
12, 21, 9, 12, 9, 24, 90, 0, 15, 25,
4, 29, 23, 25, 20, 36, 101, 15, 0, 35,
31, 41, 27, 13, 16, 3, 99, 25, 35, 0
), nrow = 10, byrow = TRUE)

# 경로의 총 거리를 계산하는 함수 정의
tsp_total_distance <- function(order) {
  # order 벡터를 정수 순열로 변환
  order <- order(order)

  # 순회 경로 생성
  tour <- c(order, order[1]) # 경로를 순회 경로로 변환 (시작점으로 돌아오기)

  # 경로의 총 거리 계산
  distance <- 0
  for (i in 1:(length(order))) {
    distance <- distance + distance_matrix[tour[i], tour[i + 1]]
  }
  return(distance) # 총 경로 길이 반환
}

# 초기 경로 설정 (numeric으로 변환)
initial_order <- as.numeric(sample(1:10)) # 초기 경로를 실수형으로 변환

# Simulated Annealing 실행
result <- GenSA(
  par = initial_order, # 초기 경로
  fn = tsp_total_distance, # 거리 계산 함수
  lower = rep(1, 10), # 도시 번호의 최소값 (실수형)
  upper = rep(10, 10), # 도시 번호의 최대값 (실수형)
  control = list(maxit = 5000) # 최대 반복 횟수
)

# 최적 경로는 실수로 나오므로 정렬하여 정수 순열로 변환
best_order <- order(result$par)

```

```
# 결과 출력
```

```
best_order # 최적 경로
```

```
result$value # 최적 경로의 거리
```